

Von der Feature-Idee zur verlässlichen Software

Matt Pocock Skills und Superpowers im Praxiseinsatz

Saburollah Safari | Projektstudie: Study Organizer

Wie lässt sich KI so in die Softwareentwicklung einbinden, dass nicht nur Code entsteht, sondern ein nachvollziehbares Ergebnis? Am Beispiel einer Moodle-nahen Kursintegration habe ich zwei Entwicklungsworkflows erprobt. Im Mittelpunkt standen Architekturentscheidungen, überprüfbare Anforderungen und die Frage, wie viel Prozess ein Feature tatsächlich braucht.

1. Ein Kurs, viele Nutzer - eine robuste Lösung

Studierende sollen einen Kurs verbinden und neue Lernaufgaben in ihrem persönlichen Planer sehen. Dahinter stehen drei Herausforderungen: Inhalte ändern sich, wiederholte Scans dürfen keine Duplikate erzeugen, und gemeinsam genutzte Kursdaten müssen von persönlichen Aufgaben getrennt bleiben.



Abbildung 1. Architekturprinzip des Mock-Features: einmal abrufen, Änderungen prüfen, berechnete Nutzer getrennt versorgen. Beispiel mit einem aufgabenfähigen Inhalt und drei Abonnenten.

Die entscheidende Trennung liegt zwischen **externer Quelle, gemeinsamer Verarbeitung und persönlicher Nutzung**. Ein Adapter vereinheitlicht PDF-, Link- und Aktivitätsinhalte. Stabile Inhalts-IDs ermöglichen den Vergleich mit dem letzten erfolgreichen Stand. Erst ein validierter Scan wird übernommen; bei einem Fehler bleiben vorhandene Daten erhalten.

Mein Beitrag: Ich traf und bewertete Produktentscheidungen, prüfte den sichtbaren Benutzerablauf und verglich die Ergebnisse. KI-Agenten unterstützten Architekturarbeit, Implementierung, Tests und Dokumentation. Die Integration wurde bewusst mit einer kontrollierten Mock-Quelle entwickelt; eine reale Moodle-Anbindung war nicht Teil des Versuchs.

2. Anforderungen und Versuchsaufbau

Die vollständigen sieben FR und sieben NFR beschreiben den gemeinsamen Vergleichskern. Der Anhang ergänzt Variantenregeln und Prüfbelege; die Matrix fasst die historischen Anforderungen nachträglich zusammen.

Funktionale Anforderungen - was die Anwendung leisten soll

ID	Anforderung	Prüfkriterium
FR-01	Unterstützten Mock-Kurs persönlich abonnieren.	Gültiger Link verbindet Kurs und eigenes Modul; unbekannter Link wird abgelehnt.
FR-02	Kurslinks auf eine stabile Identität abbilden.	Alias-Links ergeben denselben Kurs; erneute Anmeldung kein zweites Abo.
FR-03	Gemeinsamen Scan manuell starten.	Ein Abruf liefert bei einem geeigneten Inhalt drei Abonnenten je eine Aufgabe.
FR-04	Neue, geänderte und gleiche Inhalte unterscheiden.	Umbenennung erhält die ID; identischer Scan erzeugt keine zusätzliche Aufgabe.
FR-05	PDF- und Nicht-PDF-Inhalte berücksichtigen.	Fixtures decken verschiedene Arten ab; Dateiendung allein entscheidet nicht.
FR-06	Scanfehler sichtbar und ohne Datenverlust melden.	Timeout oder ungültige Antwort wird gemeldet; letzter gültiger Stand bleibt erhalten.
FR-07	Den vollständigen Ablauf in der Webapp anbieten.	Kurs verbinden, scannen und persönliche Aufgabe mit Quellenbezug öffnen.

Nichtfunktionale Anforderungen - welche Qualität dabei gelten muss

ID	Qualitätsziel	Prüfkriterium
NFR-01	Zugriffsschutz	Ohne passendes eigenes Abo sind Lesen und Scannen nicht erlaubt.
NFR-02	Datenkonsistenz	Änderungen werden vollständig übernommen oder bei Fehlern zurückgerollt.
NFR-03	Wiederholungs- und Parallelitätssicherheit	Höchstens ein Scan je Kurs gleichzeitig; gleiche Wiederholung ohne Duplikate.
NFR-04	Reproduzierbare Tests	Zeit, Quellzustand und Fehler sind ohne echten Moodle-Kurs steuerbar.
NFR-05	Datenschutz	Gemeinsame Daten und Fehlerausgaben enthalten keine persönlichen Zugangsdaten.
NFR-06	Nachvollziehbarkeit	Wichtige Entscheidungen sind mit Regel, Umsetzung und Prüfbeleg verknüpft.
NFR-07	Lokale Ausführbarkeit	Build, Typprüfung, Lint und Tests bestehen; der dokumentierte Start ist bedienbar.

Vergleichbare Ausgangsbasis, unterschiedliche Schwerpunkte

GLEICHE CODEBASIS, GETRENNTE VERSUCHE

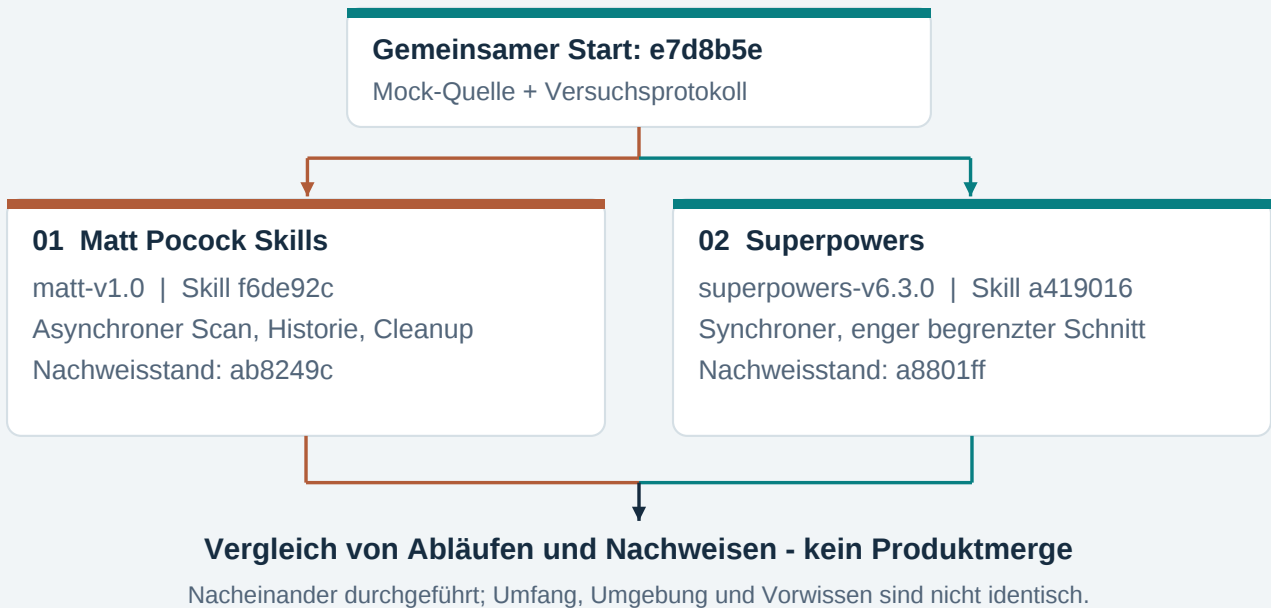


Abbildung 2. Gleicher Start, getrennte Umsetzung. Reihenfolge und Umfang begrenzen die Vergleichbarkeit.

Festgeschriebene Submodule erlaubten kontrollierte Skill-Updates. Verglichen werden Arbeitsweisen; echte Moodle-Zugänge, Polling und LLM-Erkennung waren ausgeschlossen.

Git-Fakten zu den festgeschriebenen Vergleichsständen

Git-Messwert	Matt	Superpowers
Commits ohne Merge-Commits	32	28
Geänderte Dateien insgesamt	139	92
Handgeschriebener Produktcode, hinzugefügt / entfernt	+6.555 / -121 Zeilen	+3.240 / -70 Zeilen
Branching-Strategie	Mehrere Feature-Banches und PRs	Ein isolierter Experiment-Branch

Die Werte beschreiben den Umfang, nicht die Qualität. Matt bearbeitete einen breiteren Funktionsumfang. Die Produktcode-Zeile schließt Tests, Dokumentation und generierte EF-Migrationen aus. Messbasis: gemeinsamer Start e7d8b5e, Matt ab8249c, Superpowers a8801ff; Zählregeln stehen in den Nachweisen.

3. Was die beiden Arbeitsweisen leisten

3.1 Beobachtete Arbeitsweisen

ZWEI WEGE VON DER KLÄRUNG ZUR ABNAHME

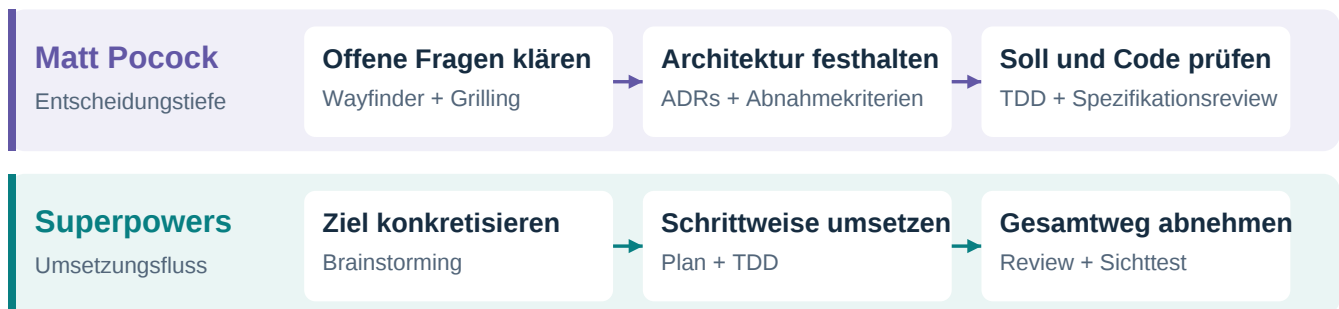


Abbildung 3. Beobachtete Schwerpunkte: Matt macht offene Entscheidungen sichtbar; Superpowers organisiert die Umsetzung. Beide nutzen Tests und Reviews.

3.2 Codeprüfung: Stärken und verbleibende Lücken

Beide Umsetzungen trennen gemeinsame Kursdaten von persönlichen Aufgaben und sichern zentrale Regeln durch Tests ab. Eine ergänzende statische Codeprüfung fand dennoch drei konkrete UI-Lücken:

Variante	Bestätigte Lücke	Verbesserung
Matt	Die Aufgabenkarte zeigt die externe Herkunft nicht an, obwohl die API sie liefert.	Quellenkennzeichnung und Link anzeigen.
Superpowers	Der Scanstatus der Kurskarte kann nach einem Scan veraltet bleiben.	Status nach Erfolg oder Fehler aktualisieren.
Superpowers	Prüfgründe aus dem Backend passen nicht zu den Übersetzungsschlüsseln.	Bezeichner vereinheitlichen und Darstellung testen.

Ein zweites KI-Modell prüfte die Befunde erneut an den festgeschriebenen Codeständen. Bei Matt betrifft die Lücke den nachträglich rekonstruierten gemeinsamen FR-07, nicht eine ausdrückliche Forderung aus Issue #77. Die beiden Superpowers-Lücken betreffen die eigene bestätigte UI-Spezifikation. Die Prüfung ist kein neuer Testlauf und kein vollständiger Qualitätsnachweis; Fundstellen und Grenzen stehen in den Nachweisen.

Technische Nachweise: Matt: 225 Backendtests, 94 Frontendtests in 20 Vitest-Dateien (Anzahl nachträglich am unveränderten Abschlussstand ermittelt) und ein Playwright-End-to-End-Test; Type-Check, Lint und Build waren erfolgreich. Superpowers: 195 Backend- und 97 Frontendtests plus Type-Check, Lint, Build und Sichtabnahme. Unterschiedliche Testbestände begründen keinen Qualitätssieger.

3.3 Persönliche Bewertung mit konkreten Gründen

Skala: **5 = sehr gut ohne relevante Einschränkung**, **4 = sehr gut mit konkreter Einschränkung**, **3 = gemischt**, **2 = eher schlecht**, **1 = sehr schlecht**. Farbig markiert ist der jeweils höhere Wert. Die Punkte beschreiben meine Erfahrung, keine objektive Codequalität.

Kriterium	Matt	Superpowers	Beobachtung hinter den Punkten
Verständlichkeit	4	5	Matt war nachlesbar, aber durch viele Fragen und Issues weniger kompakt; Superpowers führte klar und geschlossen durch den Ablauf.
Kontrolle	4	4	Beide ließen zentrale Entscheidungen zu; bei Matt erschwerten verteilte Runden den Überblick, bei Superpowers musste ich die Branch-Trennung korrigieren.
Lerngewinn	5	4	Matt vertiefte Architektur und Datenlebenszyklus; Superpowers erklärte die Anwendungsschichten gut, aber weniger tief.
Angemessener Aufwand	4	3	Matts Rückfragen waren hilfreich, aber zahlreich; Superpowers war gut strukturiert, wurde jedoch durch Kontingentpausen unterbrochen.
Vertrauen	5	4	Matt deckte durch Rückfragen Risiken auf; Superpowers bestand den Praxistest, erreichte wegen anfänglicher Start- und Konfigurationsprobleme aber keine 5.
Wiederaufnahme	4	4	Dokumente und Logs halfen bei beiden; ihre Menge erforderte dennoch erneute Orientierung.
Anpassbarkeit	4	4	Beide ließen sich anpassen; Matt blieb prozessintensiv, Superpowers erforderte einzelne manuelle Korrekturen.

Obwohl Matts Ablauf umfangreicher war, empfand ich seinen Aufwand als angemessen, weil die Rückfragen unmittelbar riskante Fachregeln klärten. Bei Superpowers belasteten dagegen wiederholte Wartezeiten den erlebten Aufwand. Kontingentpausen und lokale Startprobleme beeinflussten meine Erfahrung; sie belegen keine grundsätzliche Schwäche der Suite.

PERSÖNLICHES BEWERTUNGSPROFIL

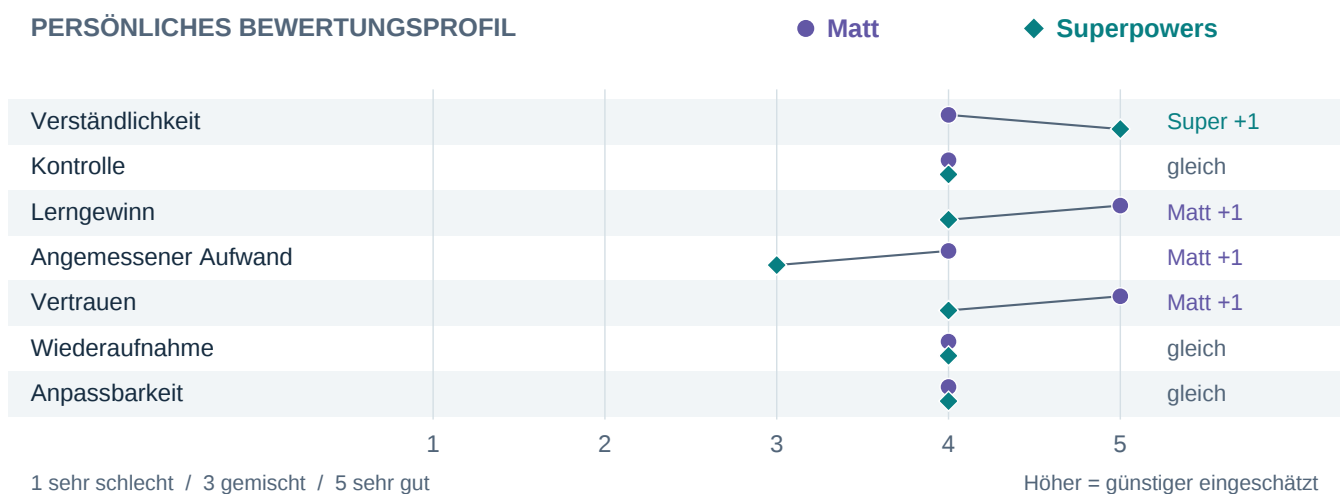


Abbildung 4. Bestätigte persönliche Bewertungen, 1 bis 5; höher ist günstiger. Keine Messung objektiver Softwarequalität.

4. Fazit: Den Prozess am Risiko ausrichten

Mein Ergebnis ist keine Rangliste, sondern eine begründete Empfehlung aus meinem Versuch: Für ein klar begrenztes Produktinkrement würde ich mit Superpowers beginnen. Sobald Fehler an Datenidentität, Berechtigungen oder Lebenszyklus schwer rückgängig zu machen sind, würde ich gezielt Matts vertiefte Architekturklärung und Spezifikationsprüfung ergänzen.

Die persönliche Bewertung stützt diese Entscheidung: Matt liegt bei Lerngewinn, angemessenem Aufwand und Vertrauen vorn; Superpowers bei Verständlichkeit. Kontrolle, Wiederaufnahme und Anpassbarkeit bewerte ich gleich. **Die weiterführende Forschungsfrage lautet daher: Lässt sich ein gemeinsamer Workflow entwickeln, der den klaren Umsetzungsfluss von Superpowers mit der Entscheidungstiefe und Absicherung von Matt verbindet, ohne gleichzeitig den Prozessaufwand beider Ansätze zu übernehmen?**

4.1 Matt: Entscheidungstiefe mit höherem Prozessgewicht

Stärke. Wayfinder, Grilling und ADRs machten fachliche Abhängigkeiten früh sichtbar. Das war bei stabiler Kursidentität und persönlichen Aufgaben besonders wertvoll: Eine falsche Regel hätte Duplikate erzeugen oder Benutzerdaten falsch zuordnen können. Die Akzeptanzmatrix verband Entscheidungen mit prüfbaren Kriterien. Der spätere Spezifikationsreview fand tatsächlich fehlende Cleanup-Regeln, obwohl das Haupt-Issue bereits geschlossen war. Damit zeigte der Ansatz einen konkreten Qualitätsnutzen über grüne Tests hinaus.

Risiko. Die vielen Klärungs- und Dokumentationsschritte vergrößerten den Prozess und passen nicht automatisch zu jeder kleinen Änderung. Entscheidungstiefe kann in Überplanung kippen, wenn risikoarme Fragen genauso ausführlich behandelt werden wie irreversible Architekturentscheidungen. Matt ist deshalb für mich besonders stark, wenn mehrere Schichten betroffen sind oder Datenregeln langfristig tragen müssen - nicht als pauschales Pflichtprogramm für jedes Feature.

4.2 Superpowers: Umsetzungsfluss im engeren Versuchsrahmen

Stärke. Brainstorming, bestätigtes Design, Plan und TDD führten klar von der Idee zum ausführbaren vertikalen Schnitt. Der zusammenhängende Weg von der Kursregistrierung bis zur sichtbaren persönlichen Aufgabe war leicht nachzuvollziehen und praktisch abnehmbar. Für einen klar definierten Umfang bietet Superpowers daher einen überzeugenden Standardprozess: kleine Schritte, unmittelbare Tests und ein sichtbares Ergebnis.

Grenze des Versuchs. Die gute Struktur garantiert weder geringe Kosten noch vollständige fachliche Abdeckung. Kontingentbedingte Wartezeiten bremsten die Ausführung; beim Wechsel auf Inline-Arbeit entfiel zudem eine unabhängige Reviewperspektive. Der Versuch umfasste bewusst keinen vollständigen Cleanup- und Reaktivierungslebenszyklus. Deshalb belegt das Ergebnis die Eignung für diesen engeren Schnitt, aber nicht, dass derselbe Ablauf Matts größeren und risikoreicheren Umfang schneller oder vollständiger geliefert hätte.

4.3 Was der Vergleich über Qualität zeigt

Grüne Tests allein genügten in keinem Versuch. Bei Matt fehlte zunächst vereinbartes Lebenszyklusverhalten; bei Superpowers scheiterte der lokale Start zunächst an Konfigurationsproblemen. Die ergänzende Codeprüfung zeigte außerdem verbliebene Lücken bei Quellenanzeige, Statusaktualisierung und Übersetzungsverträgen. Daraus folgt eine konkrete Verbesserung: UI-Tests müssen nicht nur vorhandene Elemente prüfen, sondern auch echte API-Werte und Zustandswechsel abdecken. Mein wichtigster Qualitätsmaßstab ist deshalb eine Nachweiskette aus **Regeltest, Spezifikationsreview und sichtbarer End-to-End-Abnahme**. Die zweite KI-Prüfung half, belegte Lücken von unbegründeten Fehlerbehauptungen zu trennen; sie ersetzt weder Tests noch menschliche Freigabe.

4.4 Vorschlag für einen kombinierten Workflow

Superpowers bildet den Grundablauf: Brainstorming, Design, Plan und TDD führen zügig zu einem ausführbaren vertikalen Schnitt. Vor der Implementierung folgt ein kurzer Risikocheck. Betrifft das Feature Datenidentität, Berechtigungen, Nebenläufigkeit, Datenverlust oder langfristige Lebenszyklen, werden gezielt Matts Grilling, ADRs und detaillierte Akzeptanzkriterien ergänzt. Den Abschluss bilden automatische Tests, ein unabhängiger Spezifikationsreview und ein sichtbarer End-to-End-Test. So strukturiert Superpowers den Arbeitsfluss, während Matt an risikoreichen Stellen zusätzliche Entscheidungssicherheit schafft.

4.5 Ausblick: Den kombinierten Workflow prüfen

Der kombinierte Workflow ist eine begründete Hypothese, noch kein bewiesenes Ergebnis. Ein Folgeversuch sollte dieselbe Aufgabe unter drei Bedingungen durchführen: nur Matt, nur Superpowers und der kombinierte Workflow. Verglichen werden Bearbeitungszeit, Tokenverbrauch, Anzahl der Rückfragen, Korrekturschleifen, gefundene Fehler und die persönliche Bewertung. Dadurch ließe sich prüfen, ob die Kombination tatsächlich beide Vorteile erhält oder lediglich zusätzlichen Prozessaufwand erzeugt.

Der bisherige Vergleich ist praxisnah, aber kein kontrolliertes Benchmark. Matts Umfang war breiter; Superpowers wurde später und mit mehr Domänenwissen eingesetzt. Tokenlimits begrenzten unabhängige Reviews, eine Mock-Quelle ersetzte Moodle. Diese Grenzen muss auch der Folgeversuch durch gleichen Umfang und gleiche Ausgangsinformationen kontrollieren.

5. Was ich persönlich mitnehme

Fachlich kann ich nun begründen, warum Kurs, externer Inhalt und persönliche Aufgabe getrennte Identitäten und Lebenszyklen brauchen. Methodisch lernte ich, KI-Vorschläge nicht mit Produktentscheidungen gleichzusetzen: Annahmen, Risiken und Folgen muss ich selbst bewerten.

Mein größter Lernschritt war der Wechsel vom „Code erzeugen lassen“ zum **gezielten Steuern, Begründen und Prüfen**. Ich schärfte Anforderungen, bestätigte Architekturentscheidungen und nahm das Ergebnis im Browser ab. Agenten verbesserten Tempo und Struktur; die Verantwortung für Umfang, Qualität und Freigabe blieb bei mir.

Nachweise

Die Aussagen stützen sich auf das [Versuchsprotokoll](#), die Beobachtungslogs beider Versuche und dokumentierte Test- und Reviewstände. [Methodik](#), [Variantenregeln](#) und [Quellen](#) liegen getrennt im Quellenpaket. Dort bleiben auch Versionsstände und die ausführliche Bewertungsgrundlage zugänglich.